

Project Report

ECE485/585 Final Project Group 7

Deven Bishnu, Bhavana Manikyanahalli Srinivasegowda,
Lokarjun Ramesh, Xiang Li

1. Design Specification

This project implements a last-level cache (LLC) that interfaces with 1 higher-level cache (L1) and a shared bus with other caches. The LLC may receive one of 9 possible requests from either the L1 cache or another cache on the shared bus. The project is implemented in SystemVerilog.

Valid requests to the LLC are read request (0), write request (1), read request from L1 instruction cache (2), snooped read request (3), snooped write request (4), snooped read with intent to modify request (5), snooped invalidate command (6), clear the LLC (8), and print LLC contents (9).

When using the testbench file, requests must be in the form $n \text{ address}$ where n is the number corresponding to the request, and address is the address targeted by the request (4 bytes max). These requests should be listed in a plaintext file, with 1 request per line.

When interfacing directly with the LLC, requests can be made using the functions `process_read_request_L1_DataCache()`, `process_write_request_data_cache()`, `read_request_from_L1_Instruction_cache()`, `Snooped_read_request()`, `Snooped_write_request()`, `Snooped_RWIM_request()`, `Snooped_invalidate_request()`, `clear_cache()`, and `print_cache_contents()` respectively. All functions except `clear_cache()` and `print_cache_contents()` require address as an argument.

The LLC may communicate bus operations, snoop results, and messages to the L1 cache. These communications are printed to the terminal, specifying the type of communication, the targeted address, and any other relevant information.

2. Internal Design Documentation

LLC Data structure:

- Total capacity 16MB
- 64-byte lines
- 16-way set associative
- Write allocate policy
- MESI protocol to ensure cache coherence

Field	Bits Required	Notes
Byte Offset	6	Identifies the byte within a cache line

Index Bits	14	Determines the specific cache set
Tag Bits	12	Uniquely identifies the memory block
Valid Bit	1	Indicates if the cache is valid
MESI State Bits	2	Represents MESI protocol state
LRU Replacement Bits	15	Tracks the least-recently-used order

Data Structure	Description
valid	Valid bit (1 bit)
tag	Tag bits (12 bits)
mesi_state	MESI state (2 bits: I, S, E, M)
CacheSet_t	Represents a cache set containing multiple cache lines
lines[16]	Array of 16 CacheLine_t structures
lru_state	Pseudo-LRU bits for replacement (15 bits)
Cache_t	Represents the entire cache consisting of multiple sets
sets[16384]	Array of 16,384 CacheSet_t structures
CacheStats_t	Holds cache statistics counters
read_count	Number of cache reads
write_count	Number of cache writes
hit_count	Number of cache hits
miss_count	Number of cache misses

3. Source Modules for Cache

Our cache uses one source module, `cache_simulator1`. This module is described entirely within the file `LLC_FINAL_MOD.sv`.

The module contains all of the functions mentioned in *1. Design Specification*, as well as several internal functions for initializing the cache, processing addresses into tags/indices/offsets, managing MESI states and LRU bits, and interfacing with the terminal.

4. Associated Modules Used in Validation of Cache (Including Testbench)

Our cache uses one associated module, `trace_file_reader`. This module is the testbench for `cache_simulator1`. It reads files format described in *1. Design Specification* and calls the appropriate function in `cache_simulator1`.

Our testing process used the testbench with 9 trace files, each one testing one request type.

5. Simulation Results and Usage Statistics

Our testing process used a suite of 9 trace files. The expected results are described in `README.md` in the Test Cases directory.

`testcase_0_read` (read request): Tests maximum and minimum addresses (00000000, FFFFFFFF), read miss, read hit, read miss/evict, addresses of <4 bytes, and case-insensitive addresses. It expects 29 reads, 0 writes, 10 hits, 19 misses, and a ratio of 0.345. Set 0 should contain the addresses 00200000, 00300000, 00400000, 00500000, 00600000, 00700000, 00800000, 00900000, 00A00000, 00B00000, 00C00000, 00D00000, 00E00000, 00F00000, 01000000, and 00000000. Set 3FFF should contain the address FFFFFFFF.

`testcase_1_write` (write request): Tests maximum and minimum addresses, write miss, write hit, write miss/evict, addresses of <4 bytes, case-insensitive addresses, and MESI state transitions. It expects 2 reads, 31 writes, 13 hits, 20 misses, and a ratio of 0.394. Set 0 should contain the addresses 00400000, 00500000, 00600000, 00700000, 00800000, 00900000, 00A00000, 00B00000, 00C00000, 00D00000, 00E00000, 00F00000, 01000000, 00000000, 00000002, and 01100000. Set 3FFF should contain the address FFFFFFFF. 00000002 and 01100000 should be in the M state.

`testcase_2_instruction` (read request from instruction cache): Identical to `testcase_0_read`. The LLC is a unified cache and does not distinguish instruction from data.

`testcase_3_snoopread` (snooped read request): Tests maximum and minimum addresses, MESI state transitions, addresses of <4 bytes, and case-insensitive addresses. It expects 2 reads, 1 write, 0 hits, 3 misses, and a ratio of 0.000. Set 0 should contain the addresses 00000002, 00100002, 00200002, and 00000000. Set 3FFF should contain the address FFFFFFFF. 00000002, 00100002, and 00200002 should all be in the S state.

`testcase_4_snoopwrite` (snooped write request): Tests maximum and minimum addresses, addresses of <4 bytes, and case-insensitive addresses. It expects 0 reads, 0 writes, 0 hits, 0

misses, and a ratio of 0.000. Set 0 should contain the address 00000000. Set 3FFF should contain the address FFFFFFFF.

testcase_5_snoopmodify (snooped read request with intent to modify): Tests maximum and minimum addresses, MESI state transitions, addresses of <4 bytes, and case-insensitive addresses. It expects 2 reads, 1 write, 0 hits, 3 misses, and a ratio of 0.000. All sets should have all their ways in the invalid state. There should be 1 bus operation (bus write flushwb) for address 00200002. There should be 3 invalidate commands sent to L1 for addresses 00000002, 00100000, and 00200002.

testcase_6_snoopinvalidate (snooped invalidate command): Tests maximum and minimum addresses, MESI state transitions, addresses of <4 bytes, and case-insensitive addresses. It expects 2 reads, 1 writes, 0 hits, 3 misses, and a ratio of 0.000. All sets should have all their ways in the invalid state.

testcase_8_clear (clear cache): Performs a read on the maximum and minimum addresses, then runs operation 8 (clear cache). All sets should have all their ways in the invalid state.

testcase_9_print (print cache contents): Loads in the maximum and minimum addresses, addresses in M, E, and S states, and fills all the ways of a single set. It expects 19 reads, 1 write, 0 hits, 20 misses, and a ratio of 0.000. Set 0 should contain the address 00000000. Set 1 should contain the address 00000042. Set 2 should contain the address 00000080. Set 3FFE should contain the addresses 000FFF82, 001FFF82, 002FFF82, 003FFF82, 004FFF82, 005FFF82, 006FFF82, 007FFF82, 008FFF82, 009FFF82, 00AFF82, 00BFFF82, 00CFFF82, 00DFFF82, 00EFFF82, and 00FFFF82. Set 3FFF should contain the address FFFFFFFF. 00000042 should be in the E state, 00000080 should be in the S state, and 00FFF82 should be in the M state.

6. Assumptions

Assumptions about processor: We assume that the processor will not send requests outside of our cache's address range (4 bytes).

Assumptions about memory subsystem: We assume that requests should be handled in the order they are received, and that a request will not be handled until the prior request is complete. We assume a byte offset of 6 bits is sufficient to identify a byte in the cache line. We assume the cache line is 64 bytes. We assume a write-back policy.

Assumptions about memory subsystem interfaces: We assume that addresses are no more than 4 bytes. We assume that for addresses fewer than 4 bytes wide, the leading bits are 0. We assume o

Assumptions about L1 Cache: We assume the L1 cache uses the MESI protocol.